

PIM-Accelerated Sparse Matrix-Vector Multiplication on HBM2: A Bare-Metal Hardware-Software Co-Design

Youssef Lamine

STRS Lab, Research Center

Institut National des Postes et Télécommunications (INPT)

Rabat, Morocco

Email: lamine.youssef@ine.inpt.ac.ma

Abstract—Data-driven applications are increasingly bottlenecked by the energy and latency overheads of data movement, known as the Memory Wall. Processing-in-Memory (PIM) mitigates this by relocating computation directly to the memory arrays. Building upon the PIMSys virtual prototype framework, this paper presents a hardware-software co-design for accelerating Sparse Matrix-Vector Multiplication (SpMV) on a simulated Samsung PIM-HBM architecture. We introduce a hybrid memory topology leveraging a bare-metal ARM orchestrator written in Rust, coupled with custom Very Long Instruction Word (VLIW) microcode injected into HBM2 banks. By implementing a dynamic Number of Non-Zeros (NNZ) load balancing algorithm and leveraging native memory-level parallelism across 32 banks, our architecture achieves a 5.8x speedup over a conventional sequential CPU execution baseline while entirely eliminating external bus contention.

I. INTRODUCTION

Sparse Matrix-Vector Multiplication (SpMV) is a fundamental kernel in High-Performance Computing (HPC) and deep learning, particularly for memory-bound workloads characterized by irregular data dispersion and low operational intensity. In traditional von Neumann architectures, SpMV execution suffers from severe pipeline stalling, as the CPU must fetch fragmented Compressed Sparse Row (CSR) arrays (values, column indices, and row pointers) across the system bus. This data movement paradigm—where the energy cost of a 64-bit DRAM access eclipses the cost of a floating-point operation by orders of magnitude—necessitates an architectural paradigm shift.

Processing-in-Memory (PIM), particularly Samsung’s PIM-HBM [2], offers a compelling solution by integrating Single-Instruction Multiple-Data (SIMD) floating-point units directly within the memory banks [3]. To thoroughly evaluate this architecture, we extended the gem5 [4] and DRAMSys [5] co-simulation framework. This paper details our specific contribution: a complete bare-metal hardware-software co-design targeting SpMV acceleration, featuring strict VLIW microcode alignment, dynamic load balancing, and a hybrid memory topology designed to bypass simulation limitations.

II. SYSTEM ARCHITECTURE AND HYBRID TOPOLOGY

To instantiate the PIM execution environment while avoiding the Transactional Level Modeling (TLM-2.0) limitations of bare-metal booting directly on HBM2 models, we engineered a hybrid memory topology within the gem5 environment.

A. Memory Subsystems

The system partitions the physical address space into two distinct domains:

- **Standard DDR3 Boot Space:** This memory segment securely hosts the bootloader, interrupt vector tables, and the host ARM CPU’s bare-metal firmware, ensuring deterministic system initialization.
- **HBM2 Offloading Space (via DRAMSys):** Operating as an exclusive PIM acceleration domain, this space contains the CSR matrix data structures and the injected hardware microcode.

B. Controller Tuning for SpMV

The DRAMSys controller governing the HBM2 stack is strictly configured to use a First-Ready, First-Come, First-Serve (FR-FCFS) scheduling algorithm paired with an Open-Page policy. Because SpMV sequentially accesses sparse row elements within a given bank, this configuration maximizes contiguous row buffer hits and saturates the internal Through-Silicon Via (TSV) bandwidth of the 3D memory stack.

III. HARDWARE AND SOFTWARE CO-DESIGN

Our primary contribution lies in the orchestration stack, managed within our PIMSys [1] submodule. It encompasses both the low-level Rust [6] firmware running on the host ARM processor and the custom microcode executing within the memory banks.

A. PIM Microcode (Hardware Design)

The execution units inside the HBM2 memory banks were programmed using a highly optimized, custom instruction set tailored for sparse computations:

- **Atomic MAC Execution:** Standard multi-cycle accumulation pipelines frequently suffer from Read-After-Write (RAW) data hazards. We replaced the traditional fetch-decode-execute loop with a dedicated, 1-cycle MAC (Multiply-Accumulate) hardware instruction, ensuring deterministic throughput directly at the memory cell.
- **VLIW Cache-Line Alignment:** To guarantee atomic kernel loading via a single memory burst, the microcode was structured following a strict Very Long Instruction Word (VLIW) paradigm. The kernel consists of exactly 5 active instructions padded with 27 NOP (No-Operation) instructions, forcing an exact 32-byte physical footprint that perfectly aligns with a single HBM2 cache line.

B. Host Firmware Orchestration (Software Design)

The host ARM processor, programmed in bare-metal Rust (`#![no_std]`), acts strictly as a control-plane orchestrator and performs zero matrix computations during the offloading phase.

- **NNZ-Based Load Balancing:** Due to the irregular dispersion of sparse matrices, a standard geometric row-split creates severe load imbalances ("hot banks"). We implemented a `partition_csr` algorithm that dynamically divides the matrix into 32 equal-weight computational blocks based strictly on the Number of Non-Zeros (NNZ). This ensures symmetric traffic and processing time across all 32 independent HBM2 banks.
- **Deep Sleep Orchestration:** Upon dispatching memory-mapped triggers to the HBM2 controller to initiate the All-Bank-PIM (AB-PIM) mode, the ARM core issues a Wait-For-Interrupt (`wfi`) assembly instruction. This completely suspends the processor's clock, isolating the energy consumption to the memory subsystem while it autonomously processes the SpMV calculation.

IV. EXPERIMENTAL RESULTS

We evaluated our co-design against a sequential, CPU-only baseline execution using a synthetic CSR matrix (2048x2048, NNZ=8192) within the `gem5` environment. The baseline simulates a conventional von Neumann bottleneck where the ARM processor sequentially fetches the CSR structure through the system bus.

TABLE I
PERFORMANCE COMPARISON: CPU BASELINE VS. PIM OFFLOADING

Metric	CPU Sequential	PIM Accelerated
Execution Time	239.61 s	41.42 s
External Bus Bandwidth	~1,089 B/s	~6,297 B/s
CPU Footprint (Inst.)	151,670	151,670
Load Distribution	Channel 0 Bottleneck	32-Bank Balanced

As shown in Table I, the PIM offloading architecture achieves a **5.8x speedup**, dropping execution time from 239.61 seconds to 41.42 seconds. In the CPU baseline, the identical instruction count (151,670) is accompanied by massive

pipeline stalls (approx. 240 trillion simulated idle ticks) due to memory starvation. Conversely, the PIM implementation entirely localizes the heavy data traffic to the internal HBM2 channels. The minor 6,297 B/s footprint on the external bus represents only the initial broadcast of microcode and configuration signals, successfully demonstrating the elimination of the Memory Wall for this workload.

V. CONCLUSION

This work validates the efficacy of PIM-HBM for irregular, memory-bound kernels like SpMV. By combining a hybrid memory topology, bare-metal Rust orchestration, NNZ-based load balancing, and strictly aligned VLIW microcode, we successfully bypassed the von Neumann bottleneck. The architecture leverages native 32-bank memory-level parallelism to achieve a 5.8x speedup while allowing the host CPU to enter a deep sleep state. Future work will extend this framework to process real-world asymmetric datasets from the SuiteSparse Matrix Collection and transition the validated VLIW microcode to a physical VHDL implementation for empirical power and thermal analysis. The complete source code for our `gem5` co-simulation environment [7] and the bare-metal Rust microcode [8] are available as open-source repositories.

ACKNOWLEDGMENT

The authors would like to thank the contributors of the `gem5` and DRAMSys simulation frameworks, and acknowledge the foundational virtual prototyping infrastructure provided by the original PIMSys architecture.

REFERENCES

- [1] D. Christ, L. Steiner, M. Jung, and N. Wehn, *PIMSys: A Virtual Prototype for Processing in Memory*, in The International Symposium on Memory Systems (MEMSYS '24). ACM, New York, NY, USA, 2024. [Online]. Available: <https://github.com/tukl-msd/PIMSys>
- [2] S. Lee et al., *Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology : Industrial Product*, in 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA). IEEE, Valencia, Spain, 2021, pp. 43-56.
- [3] Y.-C. Kwon et al., *A 20nm 6GB Function-In-Memory DRAM, Based on HBM2 with a 1.2TFLOPS Programmable Computing Unit Using Bank-Level Parallelism, for Machine Learning Applications*, in 2021 IEEE International Solid-State Circuits Conference (ISSCC). IEEE, San Francisco, CA, USA, 2021, pp. 350-352.
- [4] J. Lowe-Power et al., *The gem5 Simulator: Version 20.0+*, arXiv:2007.03152 [cs.AR], 2020. [Online]. Available: <https://github.com/gem5/gem5>
- [5] L. Steiner et al., *DRAMSys4.0: An Open-Source Simulation Framework for In-depth DRAM Analyses*, International Journal of Parallel Programming, vol. 50, no. 2, pp. 217-242, 2022. [Online]. Available: <https://github.com/DRAMSys/DRAMSys>
- [6] Rust Foundation, *The Rust Programming Language*, 2015. [Online]. Available: <https://www.rust-lang.org/>
- [7] Y. Lamine, *gem5-pim-spmv: PIM Offloading for SpMV Acceleration on HBM2*, GitHub Repository, 2026. [Online]. Available: <https://github.com/YoussefLe/gem5-pim-spmv>
- [8] Y. Lamine, *PIMSys: Bare-Metal Firmware and VLIW Microcode for SpMV*, GitHub Repository, 2026. [Online]. Available: <https://github.com/YoussefLe/PIMSys>